QA Test Agent: Handover Notes

Status as of 8 October 2026. This document is for whoever picks up the agent next, whether that's a person or another AI agent. It covers what the agent is, the stack, where the code lives, how to run and update it, everything changed so far, what live runs have shown, the known gaps, and the improvements still to make. The business side (who approves what, and the run flow from a user's view) is in the separate "QA Test Agent Business Workflow" document.

1. What the agent is

qa-agent takes a Redmine task ID, reads the task and its "Test Cases - ..." child (or writes cases from the description when there is none), plans the test run, and asks the user one batch of questions. After the user approves the plan, it drives the real web apps in a browser, records values and screenshots, and decides a verdict for each case. It ends with an HTML and PDF report and bug drafts that are posted only with approval.

The design principle, set by the user, is that the agent is not generic. It is agentic only where judgment is needed: planning, filling in missing steps, finding its way around new screens, verdicts and notes. Everything else is plain, fixed code: the Redmine fetch, logins, reading values, math, approval gates, the action log, and reports. Steps the agent infers are labelled, and any step that changes data needs approval.

It handles several projects. Each project is a "pack" under `packs/<slug>/`, and Savostore (customer portal plus SuiteCRM) is the first.

Current state in one line: phases 1 to 8 are built with 177 passing offline tests, and one full live run has finished (#180974: 1 PASS, 2 NOT VERIFIED, 2 AGENT INCOMPLETE, 1 SKIPPED). It works end to end, but it isn't reliable yet. Section 8 lists why.

2. Where everything is

What	Where
Source of truth	Private GitHub repo `hammad-rt/qa-agent`, branch `main`, latest commit `e60b0ca`
Copy that runs	User's laptop RT-LT-644: `/var/www/html/qa-agent` (has `.venv`, `.env`, `runs/`, `memory/`)
Assistant's working copy	Shared assistant computer: `/workspace/qa-agent` (its own `.venv`; no secrets)
Design docs	`docs/test-agent-rdp.md` (v1.3), `docs/test-agent-technical-spec.md`, `core/core-testing-spec.md`
Project pack	`packs/savostore/` (`pack.yaml`, `facts.md`, `helpers/`, `evals/`)
Template for new projects	`packs/_template/`
Run output	`runs/<task>-<date>-<time>/` on the laptop: `state.json`, `trace.jsonl`, `report.html`/`.pdf`, `bug_drafts.md`, `shots/`, `snapshots/`, `profiles/`
Memory	`memory/memory.db` (SQLite, not in git) and `memory/memory-export.json`

3. Stack

Layer	Choice	Notes
Language	Python 3.12 (laptop), 3.13 (assistant computer)	Plain venv, `requirements.txt`
Agent framework	Google ADK (`google-adk` 1.10 or later; 2.11 installed)	`LlmAgent`, `Runner`, `InMemorySessionService`, `McpToolset`
Model access	LiteLLM, model `openai/gemma4`	Through ADK's `LiteLlm` wrapper
Model server	`https://ai.rolustech.com/v1`, vLLM serving Gemma 4 31B AWQ	65,536-token context, native tool calls, no API key. It resolves **only on the Rolustech network or VPN**
Browser	Playwright MCP (`npx -y @playwright/mcp@latest`, 0.0.83)	One browser server per app, saved profile per run, headless unless `--headed`
Data models	Pydantic 2	`models.py`: plan, steps, requirements, results
Config	`pack.yaml` (PyYAML) and `.env`	No secrets in packs; passwords only in `.env`
Memory	SQLite with keyword search; optional Ollama `nomic-embed-text` embeddings	Falls back to keyword search without Ollama
Reports	Jinja2 HTML, then PDF through headless Chrome	`report/`
Redmine	REST API with `REDMINE_API_KEY` (`requests`)	`preflight/redmine.py`
Email	SMTP (Gmail app password)	Only to `REPORT_EMAIL_TO`, the user's own inbox
Offline backup model	`qwen3:4b` on local Ollama	Planned as a fallback, not wired in

Laptop limits that matter: i7-1355U, 15 GB RAM with about 4 GB free, no CUDA, and the **disk is about 96% full**.

4. How it is built

4.1 Run flow (code view)

1. `main.cmd_test` checks the model server resolves, fetches the task (`preflight/redmine.py`), and picks the pack by Redmine project ID.
2. `gate.choose_env` asks for staging, pre-prod or production. Production needs an explicit yes and is never pre-approved.
3. The planner and reviewer (`agents/planner.py`) produce a `TestPlan` as JSON. Code checks and repairs it (`plan_checks.check_plan`, `autofix`).
4. `preflight/gap_resolver.py` fills what the run needs, in this order: pack, memory, Redmine wiki, task text, then the user. Users are asked every run with the last one as the default, and passwords are prompted masked and saved to `.env` per user and environment.
5. The plan gate (`gate.plan_gate`) shows the plan and asks one batch of questions. Answers go back to the planner for a replan, and allowed action types can be pre-approved.
6. `main.run_sites` limits the browser to the hosts of this run's URLs. `login_first` offers a manual login for apps in `manual_login_apps`.
7. The coordinator (`coordinator.run_plan`) is plain code. It takes a read-only baseline, then runs cases one at a time in dependency order, splitting each case into per-app turns.
8. Each tester turn (`agents/tester.py`) is a fresh ADK session with its own Playwright MCP browser and fixed tools. Callbacks enforce site limits, risky clicks, approvals, call and time limits, redaction, page-text caps and history pruning.

9. The judge (`agents/judge.py`) sees the case brief, the action log built by code, recorded values and evidence names, but never the tester's opinions. It returns a `CaseResult`.
10. A final read-only snapshot, a settings restore check, and leftover test data from the ledger are recorded.
11. `report/builder.py` writes the HTML, PDF and JSON reports. `bug_drafts.py` writes the drafts, and `bugs --post` posts each one only after approval.
12. End of run: memory proposals wait for approval, and runs older than the retention limit are trimmed.

4.2 Modules

Module	Job
`main.py`	CLI and command flow (`check`, `test`, `rerun`, `resume`, `report`, `bugs`, `correct`, `memory`, `leftovers`, `onboard`, `pack`, `helpers`, `cases`, `eval`)
`config.py`	Settings from `.env`
`models.py`	Pydantic models: `Step`, `Requirement`, `PlannedCase`, `TestPlan`, `StepResult`, `CaseResult`, verdicts
`preflight/`	Redmine client, pack loader and validation, gap resolver, onboarding of new projects
`gate.py`	Environment choice, question batch, plan gate, production confirmation
`plan_checks.py`	Plan validation, dependency order, auto-added restore step for settings
`coordinator.py`	Case loop, retries, pause/resume, manual-login retry, baseline/final snapshots, restore pass
`agents/llm.py`	Model factory (per-role `max_tokens`), `run_agent`, JSON repair, retries, tracing
`agents/prompts.py`	Planner, reviewer, tester and judge instructions
`agents/tester.py`	Tester session, tools (`record_value`, `calculate`, `request_approval`, `mark_step`, `evidence_name`, `get_secret`, `needs_manual_login`, `finish`, `recall_lessons`, `run_helper`…), Playwright toolset, manual login window
`tools/policy.py`	Allowed hosts, risky patterns, approvals and pre-approvals, production read-only
`tools/values.py`	Money parsing and safe `calculate` (no `eval`)
`ledger.py`	Test data created or changed, settings original vs restored
`memory/`	Store, preload, recall, corrections, promotion to core, export
`helpers/`	Helper registry; core `login_form` (types the password itself) and `read_table`
`report/`	HTML/PDF report, bug drafts
`evals/`	Replay self-test against answers taken from past QA report PDFs
`casegen.py`	Writing cases from the task when there is no Test Cases child
`reliability.py`	Retries with backoff, run lock, retention, disk check
`redact.py`	Masks cards (Luhn-checked), emails, passwords and phone numbers in logs and state

4.3 Configuration

`.env` (copy `.env.example`, `chmod 600`): `LLM_BASE_URL`, `LLM_MODEL`, `LLM_API_KEY`, `LLM_MAX_CONCURRENCY`, `LLM_JSON_MODE`, `LLM_MAX_TOKENS` (8192, planner/judge), `REDMINE_URL`, `REDMINE_API_KEY`, `MAX_TOOL_CALLS_PER_CASE` (40), `MAX_MINUTES_PER_CASE` (15), memory and Ollama settings, `RUN_RETENTION`, `MIN_FREE_GB`, report and SMTP settings. Tester tuning: `TESTER_MAX_TOKENS` (2048), `TESTER_KEEP_PAGE_READS` (2), `TESTER_MAX_PAGE_CHARS` (30000), `QA_HEADED`. Passwords are added by the masked prompt as `SAVOSTORE_CRM_PASSWORD__<USER>` style names. Never type them into files by hand, and never commit `.env`.

`packs/savostore/pack.yaml`: Redmine project ID 1567, wiki page with staging credentials (read for addresses only, never copied), apps `portal` (staging `https://staging5.rolustech.com:4225/`) and `crm` (staging `https://staging5.rolustech.com:4220/`), production URLs, accounts with password env names and role markers, `ask_accounts_each_run: true`, `manual_login_apps: [portal]`, known sites, pre-approvable and always-ask actions, risky click patterns, memory areas, and helper settings (all project helpers are still empty, so they're off).

5. Running, testing and updating

```
.venv/bin/python -m agent_tester check
.venv/bin/python -m agent_tester test 180974 [--env staging] [--cases TC1,TC3] [--dry-run] [--headed]
.venv/bin/python -m agent_tester resume runs/<run folder> [--headed]
.venv/bin/python -m agent_tester rerun 180974 --cases TC4,TC5      # reuses the last approved plan
.venv/bin/python -m agent_tester report runs/<run folder>
.venv/bin/python -m agent_tester bugs runs/<run folder> [--post]
.venv/bin/python -m agent_tester correct runs/<run folder> TC4 PASS "why"
.venv/bin/python -m agent_tester eval --project savostore [--live]
.venv/bin/python -m unittest discover -s tests -t .               # 177 offline tests
```

Before a run: the laptop must be on the Rolustech network or VPN (`getent hosts ai.rolustech.com` must print an address). It needs `REDMINE_API_KEY` in `.env` and a screen for the manual login window.

How updates reach the laptop (it has no GitHub credentials):

13. On the assistant computer, in `/workspace/qa-agent`: commit, then `git push origin main` (after `gh auth setup-git`; pipe push output through a token-redacting `sed`), and check with `git ls-remote origin main`.
14. `git bundle create /workspace/qa-update.bundle <previous commit>..main`, then copy it to the laptop as `/home/hammad.iqbal/qa-update.bundle`.
15. On the laptop: `git pull --ff-only /home/hammad.iqbal/qa-update.bundle main && git update-ref refs/remotes/origin/main HEAD`, delete the bundle, and run the tests.

6. Changes made (commit history)

All commits are dated 6 October 2026. The design docs were written from 1 October, and the code was first built on 1 October and rebuilt into this repo on 6 October.

Commit	Change
`3985ed9`	Phase 1: skeleton, Redmine fetch, pack loading, gap resolver, action policy, coordinator, ADK planner/reviewer/tester/judge, CLI (`check`, `test --dry-run`, `resume`), offline tests
`4420ce6`	Phases 2 and 3: SQLite memory (preload, recall, corrections, promotion to core) and safety (read-only snapshots, test-data ledger, settings restore checks, retries, run lock, retention, masked tracing)
`606126a`	Phase 4: full HTML/PDF report, bug drafts with approval-gated posting, known bugs still present, `--cases`/`--retest-bug`/`rerun`, progress log, own-inbox email; Luhn check so run IDs aren't masked as cards
`2ca4962`	Phase 5: replay self-test that scores the judge against answers from past report PDFs (90% agreement bar, no FAIL judged PASS)
`cc060dc`	Phase 6: helpers registry, `login_form` (password typed by code), `read_table`, `read_wallet` and five Savostore data-changing helpers (approval-gated, off until UI labels are confirmed)
`b2e9a57`	Phase 7: more projects: pack template, `pack new/validate`,

	`onboard` from Redmine, isolation tests with a fictional pack
`ea1f596`	Phase 8: write cases from the task description when there's no Test Cases task, with code checks that each expected result quotes the task; `cases --post` after approval
`c8b0f3e`	JSON mode for planner/reviewer/judge, repair of small JSON slips, 3 correction tries, clean stop instead of a traceback; a judge failure becomes AGENT INCOMPLETE
`66bd9c4`	`max_tokens` raised to 8192 (the server default of 512 cut plans off); exact JSON shapes in prompts
`928b140`	First live-run fixes (#180966): environment and users asked every run, per-user/per-environment passwords, no crash when pre-approving settings changes, restore pass, replan with gate answers
`b84ecd3`	Browser allowlist included ports (fixed `ERR_BLOCKED_BY_CLIENT` on staging5:4225/4220); run URLs used for pre-prod
`db5c4d6`	Browser sites come from each run's URLs, and any other site (like a payment page) needs the user's OK; Playwright `--allowed-origins` removed
`a048896`	#180974 fixes: saved browser profile per app per run, manual login for CAPTCHA with retry or BLOCKED, tester history pruning, page-text cap, tester reply limit 2048, context errors no longer pause the run, planner rules for status changes and record ownership
`fb9fb02`	Clear "model server not reachable, check VPN" message instead of a traceback; checked before planning
`e60b0ca`	Manual login window now actually visible (`DISPLAY`/`WAYLAND_DISPLAY` passed to the browser server), check that the login left the form, `--headed` option

7. Live runs so far

Run	What happened	Outcome
#180966 (6 Oct)	7-case plan built and approved; crashed with `PolicyViolation` when pre-approving `change_settings`	Fixed in `928b140`; not rerun since
#180974, 13:49	Portal login hit reCAPTCHA on TC1 to TC4 (AGENT INCOMPLETE); TC5 crashed on the 65,536-token context limit	Fixed in `a048896`
#180974, 14:05 and 14:09	Model server unreachable (VPN dropped), then manual-login window invisible	Fixed in `fb9fb02` and `e60b0ca`
#180974, 14:20	**First finished run.** Manual portal login worked and lasted the run	TC3 PASS, TC1 and TC2 NOT VERIFIED, TC4 and TC5 AGENT INCOMPLETE, TC6 SKIPPED

What the finished run showed:

- **TC3 PASS:** it clicked order A1148, and the URL had `editedOrder=A1148` as expected.
- **TC1 NOT VERIFIED:** it took a screenshot but never recorded which row had a null order name or what label it showed.
- **TC2 NOT VERIFIED:** no evidence was captured.
- **TC4 AGENT INCOMPLETE:** the SuiteCRM search for order 1148 kept returning "You currently have no records saved", and it ran out of tool calls.
- **TC5 AGENT INCOMPLETE:** it changed an item status to Purchased, then ran out of tool calls before saving the package status change.

- **TC6 SKIPPED:** the check needs the API response, which the UI alone can't prove.

8. Known gaps and issues (most important first)

16. **Screenshots land in the wrong folder.** `evidence_name` returns a bare file name, and Playwright MCP saves it in the folder the command was run from (the repo root on the laptop: `tc1-01-activity-list-showing-order-numbers.png`, `login-crm-admin.png`), not in `runs/<run>/shots/`. The report and judge then can't find the evidence. Fix: return an absolute path under the run's `shots/` folder, and have code check the file exists.
17. **Testers don't record what they saw.** Verification steps end with a screenshot but no `record_value` or observation, so the judge can't confirm and says NOT VERIFIED. Fix: require one recorded observation per verify step (enforced in code before `mark_step` succeeds), and give the judge a short excerpt of the last page text.
18. **SuiteCRM navigation is weak.** The tester can't find orders by number (wrong search field or format: `A1148` vs `1148`). All Savostore helpers are off because their page paths and labels were never filled in. Fix: fill in the `helpers:` settings in `pack.yaml` from a live session and turn on `crm_move_order_status`, `crm_add_parcel` and `crm_ship_parcel`, add an "open record by number" helper, or look up record IDs read-only through the existing savostore SuiteCRM connector and open them by URL.
19. **The tool-call budget is too small for CRM cases.** 40 calls per case isn't enough for three status changes. Fix: scale the budget with the number of steps, or give each step its own budget.
20. **A CAPTCHA login is needed every run.** The profile is per run, so the user logs in by hand each time. Fix: keep one profile per app, environment and user across runs, with an expiry check. The cleanest fix is for the developers to turn off reCAPTCHA on staging5 or use Google's test keys.
21. **No API-level checks.** Cases like TC6 are skipped. Playwright MCP already has `browser_network_requests` and `browser_network_request`, so a read-only "check the API response" step could cover them.
22. **Memory is still tight.** Pruning keeps the last 2 page reads and cuts each at 30,000 characters, but nothing counts tokens or lowers the reply limit near 65,536. Fix: count tokens before each call and lower `max_tokens` or summarize when close.
23. **The judge's accuracy is unproven.** `eval --live` has never been run, so the 90% agreement bar hasn't been checked against the 28 Sep Savo Rewards report.
24. **Only one task has been run live.** #180966 hasn't been rerun since its fix, and Phase 8 (cases written from the description), the bug posting flow, email, onboarding, memory promotion and helpers haven't been used live.
25. **Pre-prod URLs are unknown.** The agent asks for them the first time pre-prod is picked.
26. **It only works on the office network.** The model server only resolves there, and the offline backup model isn't wired in.
27. **Housekeeping:**

    - The README "Status" section is out of date (it still says 121 tests and no live run).
    - Stray screenshots sit in the laptop repo root.
    - Each run keeps a Chrome profile (about 11 MB) on a disk that is about 96% full.
    - `runs/` retention is 5 full runs.

28. **Speed:** one case at a time, about 1 to 4 minutes per case, one run at a time (lock file).

9. Improvements still to make (suggested order)

29. Fix evidence paths and require recorded observations (gaps 1 and 2). This alone should turn most NOT VERIFIED results into real verdicts.

30. Make SuiteCRM reliable: fill in the helper settings from a live session, add "open record by number", and consider read-only lookups through the savostore connector.
31. Scale the tool-call budget per step, and add a token-aware reply limit.
32. Keep logins across runs (one profile per app, environment and user, with expiry checks), and ask the developers about reCAPTCHA on staging.
33. Run #180974 again, then #180966 and #165348, and a task with no Test Cases child to try Phase 8. Run `eval --live` and record the score.
34. Add read-only network checks for API-level cases.
35. Try the post-run steps live: bug drafts and `bugs --post` (one approval each), email to the user's own inbox, memory approvals.
36. Add pre-prod URLs once known, and onboard a second project with `onboard`.
37. Optional: wire in the offline backup model for planning only, run independent read-only cases in parallel, and add an end-of-run cleanup for profiles.
38. Update the README "Status" section after each milestone.

10. Rules the next person must keep

- No passwords in code, packs, memory, reports or chat. Secrets go only through the masked prompt into `.env`, and memory stores only env var names. The Redmine wiki has plaintext staging credentials: read addresses only, and never copy secrets from it.
- The model never sees passwords. `login_form` and `get_secret` type them from env vars, and only into the login form.
- Every data-changing, money-moving, settings, posting or email step needs approval. Settings changes, Redmine posts, emails and production writes are always asked one by one and never pre-approved. Production is read-only unless approved.
- Bug drafts and generated test cases are posted to Redmine only after the user approves each one. Report emails go only to the user's own inbox unless they confirm another address.
- The browser stays on the run's sites, and any other site needs the user's OK. CAPTCHAs are never solved by the agent. A person logs in, or the case is BLOCKED.
- Status changes on records aren't settings and aren't reverted. Use test records made for the run, owned by the run's chosen user, and log them in the ledger.
- Don't install anything on the user's laptop unless asked (the user set up the venv). Don't build new features unless asked; doc and spec updates are fine.

11. Practical notes for an agent taking over

- The laptop has no `rg` (use `grep`) and no Python `playwright` package. Scripts that import `agent_tester` need `PYTHONPATH=.`.
- The assistant computer can't reach `ai.rolustech.com` or staging5, so live runs only happen on the laptop. Offline tests run anywhere.
- The old `/var/www/html` on the assistant computer is gone, so the working copy is `/workspace/qa-agent`, cloned with `gh repo clone` (the token comes from the environment; never print it).
- The GitHub connector in the assistant app isn't connected, so push with `git` and `gh` from the assistant computer.
- Every tester turn is a fresh ADK session, so context problems happen inside one turn, usually one CRM turn with big SuiteCRM pages.

- When a run fails, start with `runs/<run>/state.json` (results, checkpoints, `manual_logins`) and `trace.jsonl`.
- Related helpers: the "test tasks savostore" bot holds the Savostore QA knowledge (test cases, past reports, the business workflow document), and the "MR Review bot" is unrelated (GitLab merge-request audits).